

SE1 Summative 2 Reviewer: Modules 3 and 4

CS0025 Software Engineering 1. Combined study notes for Module 3 (Agile Methodologies) and Module 4 (System Models and Requirements). The blue boxes are memory aids I made; they are not from the slides.

What Is Inside

- **Part 1, Module 3:** Agile and Scrum, then Crystal, DSDM, FDD, Lean, and XP.
- **Part 2, Module 4:** system models, DFDs, system requirements, and fact-finding techniques.
- Each part ends with a one-page cram sheet. Read the cram sheets last, right before the exam.

Fast Memory Map

Topic	Hook
Agile values	People, Product, Partners, Pivot
Scrum times (2-week sprint)	Planning 4 h, Daily 15 min, Review 2 h, Retro 1.5 h
Scrum events	Some Pandas Dance, Rest, Relax, Repeat
DSDM phases	Pam Finds Big Fat Dogs In Parks
FDD practices	Model it, Own it, Check it, Show it
Lean principles	Even A Dull Day Earns Bright Optimism
XP phases	Plan And Design, Execute, Wrap, Close
Modeling kinds	FABE: Functional, Architecture, Business, Enterprise
DFD symbols	Every Person Files Stuff: Entity, Process, Flow, Store
Fact-finding	I Read Questions, Observe, Research

Part 1: Module 3, Agile Methodologies

How To Use These Notes

Module 3 has two chapters. **Chapter 1** is Agile in general and Scrum. **Chapter 2** is five other Agile methods: Crystal, DSDM, FDD, Lean (LSD), and XP. The exam likes lists, so the memory aids matter most for: the four values, the Scrum roles and events, the DSDM phases, the FDD practices, and the Lean principles.

Method	One-Line Identity	Hook To Remember
Agile	Iterative, customer-focused way of building software	Small steps, feedback, change is welcome
Scrum	Agile framework built on sprints and a few roles	Sprints, backlog, daily 15 minutes
Crystal	Agile that puts people first, by Alistair Cockburn	People over process
DSDM	Agile framework with fixed time and budget	Timebox, MoSCoW, prototype
FDD	Build it feature by feature	Feature is the unit of work
Lean (LSD)	Agile from Toyota lean manufacturing	Kill waste, just in time
XP	Short releases for changing requirements	Customer in the target

Chapter 1: Agile and Scrum

What Is Agile?

Agile is a group of approaches where requirements and solutions **evolve** through teamwork between **self-organizing, cross-functional teams** and their customers or end users. It pushes four things: **adaptive planning, evolutionary development, early delivery, and continual improvement**, and it welcomes quick, flexible response to change. **Scrum is one type of Agile.**

Agile breaks the work into **small increments** with little up-front planning and design. At the end of each iteration a **working product is shown to stakeholders**, which lowers risk and lets the product adapt quickly. Agile touches every function: planning, analysis, design, coding, testing.

The Four Agile Values

We Value This More	Over This	What It Really Means
Individuals and interactions	Processes and tools	Tools matter, but competent people working together matter more.
Working software	Comprehensive documentation	Documentation helps, but the point of development is software, not paperwork.
Customer collaboration	Contract negotiation	A contract is no substitute for working closely with customers to

		find what they need.
Responding to change	Following a plan	A plan must not be so rigid that it cannot absorb new technology, priorities, or understanding.

Memory Aid

The four values, left side only: **People, Product, Partners, Pivot.**

People (individuals), Product (working software), Partners (customer collaboration), Pivot (responding to change).

Trap: Agile does not say the right side is worthless. It says the left side is valued **more**.

The Twelve Agile Principles

The slides give them in three groups of four. Learn them as the idea behind each one:

1. Customer satisfaction through **early and continuous delivery** of valuable software.
2. **Welcome changing requirements**, even late in development.
3. Deliver working software **frequently** (weeks rather than months).
4. **Close, daily cooperation** between business people and developers.
5. Build projects around **motivated individuals** and trust them.
6. **Face-to-face conversation** is the best communication (co-location).
7. **Working software** is the primary measure of progress.
8. **Sustainable development**: keep a constant pace.
9. Continuous attention to **technical excellence** and good design.
10. **Simplicity**: the art of maximizing the work not done.
11. Best architectures, requirements, and designs come from **self-organizing teams**.
12. Regularly **reflect** on how to become more effective, then adjust.

Memory Aid

Chunk the twelve into three stories of four:

Customer story (1 to 4): satisfy, welcome change, deliver often, talk daily.

Team story (5 to 8): motivated people, face to face, software is progress, steady pace.

Craft story (9 to 12): excellence, simplicity, self-organizing, reflect and adjust.

Agile Vs Waterfall

Waterfall flows **sequentially** from start to end. Agile is **incremental and iterative**. The customer sees a waterfall product only at the end, but sees an Agile product early and often.

Point	Agile	Waterfall
Flow	Incremental and iterative	Sequential, start to end
Structure	Broken into individual models designers work on	Not broken into individual models
Customer sees product	Early and frequently, can make changes	Only at the end of the project

Structure feel	Considered unstructured compared to waterfall	More secure because it is plan oriented
Project size	Small projects done quickly; large projects are hard to estimate	All sorts of projects can be estimated and completed
Errors	Can be fixed in the middle of the project	Whole product tested only at the end; a requirement error can force a restart
Iteration	Short iterations of 2 to 4 weeks, very little planning	Big phases; each phase ends with a detailed description of the next
Documentation	Less priority than software	Top priority, even used to train staff or hand over to another team
Testing	Every iteration has its own testing phase, so regression testing each release	Only after the development phase
Delivery	Shippable features at the end of each iteration	All features delivered at once after a long implementation
Testers and developers	Work together; user acceptance at the end of every sprint	Work separately; user acceptance at the end of the project
Developers in planning	Close communication; developers help analyze requirements	Developers not involved in requirements and planning; delays between tests and coding

Memory Aid

Waterfall = one big **waterfall**, you only see the lake (finished product) at the bottom.

Agile = many small **agile** steps, you see something after every step.

Quick test: if the statement says "only at the end" or "top priority documentation", it is Waterfall.

What Is Scrum?

Scrum is an **agile process framework for managing complex knowledge work**, first aimed at software development, research, and advanced technologies. It is designed for **teams of ten or fewer**, who split work into goals finished in **timeboxed iterations called sprints**. A sprint is **no longer than a month** and most commonly **two weeks**. Progress is tracked and re-planned in **15-minute timeboxed daily meetings called daily scrums**.

Regular project management builds the whole product in one pass. Scrum delivers **several iterations** of a product so stakeholders get the **highest business value in the least time**. It asks for frequent planning and goal setting, which keeps the team focused and productive.

Benefits Of Scrum (Seven)

- Flexibility and adaptability
- Creativity and innovation
- Lower costs

- Quality improvement
- Organizational synergy
- Employee satisfaction
- Customer satisfaction

Memory Aid

Seven benefits, in a 2 + 2 + 3 pattern:

Ways of working: Flexibility, Creativity.

Business results: Lower costs, Quality.

Who is happy, inside out: Organization (synergy), Employees, Customers.

The Three Scrum Roles

Role	What They Do	Key Words
Scrum Master	Facilitator of the scrum process. Holds the daily meetings, makes sure scrum rules are enforced, coaches and motivates the team, removes impediments , and gives the team the best conditions to deliver.	Facilitator, removes impediments, enforces the rules
Product Owner	Represents the stakeholders (usually customers). Sets product expectations, records changes, and administers the backlog , a constantly updated to-do list. Prioritizes goals for each sprint by value to stakeholders.	Stakeholders, backlog, prioritizes
Scrum Development Team	A self-organized group of three to ten people with the business, design, analytical, and development skills to do the work. They self-administer tasks and are jointly responsible for each sprint goal.	3 to 10, self-organized, does the work

Memory Aid

Master = Manages the process, not the product. The Scrum Master never decides what to build.

Owner = Owns the list. The Product Owner owns and orders the backlog.

Team = Three to ten doers. (Some slides say teams of ten or fewer overall; the team itself is 3 to 10.)

The Six Scrum Events

Order on the slide: Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective, Backlog Refinement.

Memory Aid

Sentence: Some **Pandas Dance, Rest, Relax, Repeat.**

= **Sprint, Planning, Daily, Review, Retrospective, Refinement.**

Event	What Happens	Time For A 2-Week Sprint
Sprint	Basic unit of development. A timebox with a length fixed in advance, between one week and one month (two weeks most common).	2 weeks typical
Sprint Planning	Team agrees on scope, selects backlog items that fit one sprint, prepares the sprint backlog , and agrees on a sprint goal (a short description of what they forecast to deliver). Tasks are forecast, usually by voting.	4 hours
Daily Scrum	Starts precisely on time even if members are missing; same time and place every day. Anyone may attend, but only the development team contributes.	15 minutes
Sprint Review	Team reviews completed and not-completed work, presents completed work to stakeholders, and collaborates on what to do next. Incomplete work cannot be demonstrated.	2 hours
Sprint Retrospective	Team reflects on the past sprint and agrees on improvement actions. Facilitated by the Scrum Master.	1.5 hours
Backlog Refinement	Ongoing; formerly called grooming . Reviews backlog items so they are clear and ready for future sprints. Items may be split, acceptance criteria clarified, dependencies found.	Ongoing

Memory Aid

Times for a two-week sprint: **4 - 15 - 2 - 1.5**.

Planning **4 hours** (longest, it sets the whole sprint), Daily **15 minutes**, Review **2 hours**, Retro **1.5 hours**.

Review and Retro are the two "look back" meetings and Retro is the shorter one.

Trap: Review is **with stakeholders** and shows **finished work only**. Retro is **the team alone**, about the **process**.

The Three Daily Scrum Questions

1. What did I complete **yesterday** that helped the team meet the sprint goal?
2. What do I plan to complete **today** toward the sprint goal?
3. Do I see any **impediment** that could block me or the team?

Memory Aid

Y-T-B: Yesterday, Today, Blockers.

The Three Retrospective Questions

1. What **went well** during the sprint?
2. What **did not go well**?
3. What **could be improved** for better productivity next sprint?

Memory Aid

Well, Not Well, Better. (Keep, Stop, Improve in plain words.)

Sprint Planning Checklist

- Mutual discussion and agreement on the scope of work
- Select product backlog items that can be completed in one sprint
- Prepare a sprint backlog with the work needed
- Agree on a sprint goal
- Four hours for a two-week sprint
- Forecast tasks, usually by voting

Chapter 2: Other Agile Methods

Crystal

Crystal is an Agile approach that focuses on **people and their interactions** rather than processes and tools. It was developed by **Alistair Cockburn**, who believed people's skills, talents, and the way they communicate have the biggest impact on a project's outcome.

Two beliefs: teams can **streamline their own processes** as they work and become more optimized, and **projects are unique and dynamic**, so they need specific methods.

Phase	What Happens
1. Chartering	Create the development team, do a preliminary feasibility analysis, develop an initial plan, fine-tune the methodology.
2. Cyclic Delivery	Two or more delivery cycles: update and refine the release plan; implement a subset of requirements through one or more program-test-integrate iterations; deliver the integrated product to real users; review the project plan and methodology.
3. Wrap Up	Deployment into the user environment, post-deployment reviews and reflections.

Memory Aid

Crystal phases: **Charter, Cycle, Wrap**. Start a club (charter), run the meetings in cycles, wrap up.

Crystal = Cockburn = people. Both start with a hard C sound.

DSDM (Dynamic Systems Development Method)

DSDM is an Agile **project delivery framework**. Users must be **actively involved**, teams are **given the power to make decisions**, and the focus is **frequent delivery** of the product.

Three Techniques

Technique	Meaning
Timeboxing	Split the project into portions, each with a fixed budget and delivery date . Requirements are prioritized per portion. Because time and budget are fixed, the only variable is the requirements : if time or money runs short, the lowest-priority requirements are dropped .
MoSCoW	A prioritization technique to agree with stakeholders how important each requirement is. Must have, Should have, Could have, Won't have .
Prototyping	Build prototypes early so shortcomings are found early and future users can test-drive the system. Gives good user involvement , a key DSDM success factor.

Memory Aid

MoSCoW is an acronym with filler letters. The capitals carry the meaning: **M**ust, **S**hould, **C**ould, **W**on't.

Priority falls in that order. Won't means not this time.
Timeboxing in one line: **time and money are fixed, scope bends.**

The Seven DSDM Phases

1. Pre-project
2. Feasibility Study
3. Business Study
4. Functional Model Iteration
5. Design and Build Iteration
6. Implementation
7. Post-project

Memory Aid

Sentence: **Pam Finds Big Fat Dogs In Parks.**

= **P**re-project, **F**easibility, **B**usiness study, **F**unctional model, **D**esign and build, **I**mplementation, **P**ost-project.

Shape: two bookends (Pre and Post), two studies in front (Feasibility, Business), two iterations in the middle (Functional, Design and Build), then Implementation.

FDD (Feature Driven Development)

FDD is built around **designing and building features**. Unlike other Agile methods it describes very **specific, short phases of work done separately per feature**. The slide names these activities: domain walkthrough, design inspection, promote to build, code inspection, and design.

The Eight Practices

1. Domain Object Modeling
2. Development by Feature
3. Component or Class Ownership
4. Feature Teams
5. Inspections
6. Configuration Management
7. Regular Builds
8. Visibility of Progress and Results

Memory Aid

Chunk as **1 + 3 + 3 + 1: Model it, Own it, Check it, Show it.**

Model it: Domain Object Modeling.

Own it (three): Development by Feature, Class Ownership, Feature Teams.

Check it (three): Inspections, Configuration Management, Regular Builds.

Show it: Visibility of progress and results.

Lean Software Development (LSD)

Lean translates **lean manufacturing** principles to software. It is adapted from the **Toyota Production System** and built on "**Just in time production**". It aims to **increase speed and decrease cost**.

The Seven Lean Principles

#	Principle (Two Names)	Meaning
1	Eliminating Waste	If an activity can be skipped, or the result reached without it, it is waste. Examples: abandoned partial code, paperwork, extra features customers rarely use.
2	Amplifying Learning	Development is continuous learning through iterations. Software value is measured in fitness for use , not conformance to requirements.
3	Defer Commitment (Decide as late as possible)	Uncertainty is normal, so delay decisions until they can be based on facts, not assumptions .
4	Early Delivery (Deliver as fast as possible)	The sooner a defect-free product ships, the sooner feedback comes. "Not the biggest that survives, but the fastest."
5	Empowering the Team	Managers learn to listen to developers. Follows the Agile principle: build around motivated people and trust them.
6	Building Integrity	The customer has a whole-system experience: how it is advertised, delivered, deployed, accessed, how intuitive it is, its price, how well it solves the problem.
7	Optimize the Whole	Defects pile up as work is split into parts, so find and remove root causes. Systems are the product of their interactions , not just the sum of parts. "Think big, act small, fail fast; learn rapidly."

Memory Aid

Sentence: **Even A Dull Day Earns Bright Optimism.**

= **E**liminate waste, **A**mplify learning, **D**efer commitment, **D**eliver early, **E**mpower team, **B**uild integrity, **O**ptimize whole.

Number 1 is the most fundamental: **Eliminate waste.**

Seven Wastes Of Software Development (Cisco Version)

The DevNet module (see the NetComms notes) names the seven wastes. Useful if a question asks "which is a waste":

- Partially done work
- Extra processes
- Extra features
- Task switching
- Waiting

- Motion
- Defects

Memory Aid

Please **Eat Every Tasty Waffle, My Darling** = Partial work, Extra processes, Extra features, Task switching, Waiting, Motion, Defects.

XP (Extreme Programming)

XP is very helpful when **requirements change constantly** or the customer is **unsure about the functionality**. It pushes **frequent releases in short development cycles**, which improves productivity and gives checkpoints where customer requirements are easy to add. **XP develops software keeping the customer in the target.**

The Six XP Phases

Phase	Activities
Planning	Identify stakeholders and sponsors; infrastructure requirements; security information; Service Level Agreements and conditions
Analysis	Capture stories in the " parking lot "; prioritize them; scrub stories for estimation; define the time iteration; plan resources for development and QA
Design	Break down tasks; prepare test scenarios for each task; regression automation framework
Execution	Coding; unit testing; manual test scenarios; defect reports; convert manual to automated regression tests; mid-iteration review; end-of-iteration review
Wrapping	Small releases; regression testing; demos and reviews; new stories as needed; process improvements from the end-of-iteration review
Closure	Pilot launch; training; production launch; SLA guarantee assurance; review SOA strategy; production support

Memory Aid

Phases: **P-A-D-E-W-C** = **Plan And Design, Execute, Wrap, Close.**

Parking lot = where user stories wait to be prioritized (Analysis).

Story Cardboard = sticky notes on a board (manual, slower). Online Storyboard = the same stories kept in an online tool that several teams can share.

One-Page Cram Sheet

Topic	Remember
Agile values	People, Product, Partners, Pivot (left side valued more)
Scrum roles	Scrum Master (process, removes blocks), Product Owner (backlog, stakeholders), Dev Team (3 to 10)
Scrum events	Sprint, Planning, Daily, Review, Retro, Refinement
Times (2-week sprint)	Planning 4 h, Daily 15 min, Review 2 h, Retro 1.5 h
Daily questions	Yesterday, Today, Blockers
Retro questions	Well, Not well, Better
Crystal	Cockburn; Charter, Cycle, Wrap
DSDM	Timebox, MoSCoW, Prototype; 7 phases PFBFDIP
FDD	8 practices: Model, Own (3), Check (3), Show
Lean	7 principles: Eliminate waste first; Toyota; just in time
XP	Changing requirements; 6 phases PADEWC; parking lot

Part 2: Module 4, System Models and Requirements

How To Use These Notes

Module 4 has two chapters. **Chapter 1** is modeling: system models, architecture, business process models, design diagrams, and **data flow diagrams (DFDs)**. **Chapter 2** is system requirements and the **fact-finding techniques** used in requirements analysis. Expect to be asked to name symbols, name the kinds of models, and match each fact-finding technique to its description.

Chapter 1: System Models

What Is A Model?

A **model** is a description where details are removed in a systematic way and simplified so it is easy to understand. **Modeling a system** means identifying its **characteristics, states or statuses, and behavior** using a notation. **Systems modeling** is the interdisciplinary study of using models or diagrams to conceptualize and construct systems in business and IT development.

The Four Kinds Of Modeling

- Functional Modeling
- System Architecture
- Business Process Modeling
- Enterprise Modeling

Memory Aid

F-A-B-E: Functions Always Benefit Enterprises.

Order goes from small to big: a **function**, the whole **architecture**, the **business** processes, the entire **enterprise**.

Functional Modeling

A structured representation of the **functions, activities, actions, processes, and operations** inside the system. It is a **graphical representation of an enterprise's function within a defined scope**. Purposes: describe functions and processes, help discover information needs, help identify opportunities, and set a basis for working out product and service costs.

The functional perspective is one perspective of business process modeling; the others are **behavioural, organizational, and informational**. It focuses on the **dynamic process**. The main concept is the **process**: a function, transformation, activity, action, or task.

Four Basic Elements Of A Functional Model

Element	Meaning	Shape
---------	---------	-------

Process	Transformation from input to output	Circle or square with a number sequence
Store	A data collection or material	Rectangle, usually with the right side open
Flow	Movement of data or material	Arrow or flow line
External Entity	Outside the modeled system but interacts with it	Rectangle

Memory Aid

P-S-F-E: Please Stop Fighting, Everyone = Process, Store, Flow, External entity.

Functional Block Diagram And FFBD

- **Functional block diagram:** describes the functions and interrelationships of a system. It shows functions as **blocks**, inputs and outputs as **lines**, the relationships between functions, and functional sequences and paths for matter or signals.
- **Functional Flow Block Diagram (FFBD):** a **multi-tier, time-sequenced, step-by-step** flow diagram of the system's functional flow. It defines detailed operational and support sequences and is widely used in software development. Flow steps may combine hardware, software, personnel, facilities, and procedures.

Memory Aid

FFBD = **Functional Flow**: the word **Flow** means time-sequenced steps. A plain functional block diagram is just blocks and lines.

System Architecture

The **conceptual model** that defines the **structure, behavior, and more views** of a system. An architecture description is a **formal description and representation** of a system, organized to support reasoning about its structures and behaviors. A system architecture can consist of **components and sub-systems** that work together to implement the overall system.

Business Process Modeling (BPM)

BPM is the activity of representing the processes of an enterprise so the current process can be **analyzed, improved, and automated**. Typical business objectives: increase process **speed** or reduce cycle time; increase **quality**; reduce **costs** (labor, materials, capital). A decision to invest in it is often motivated by the need to document requirements for an IT project.

Business Model

A framework for creating **economic, social, or other forms of value**. It represents the core aspects of a business: purpose, offerings, strategies, infrastructure, organizational structures, trading practices, and operational processes and policies.

Business Process And Its Three Types

A **business process** is a collection of related, structured activities or tasks with a goal to produce a specific service or product for a particular customer.

Type	What It Does	Examples From The Slide
Management Process	Governs system operation	Corporate governance, strategic management
Operational Process	The core business ; creates the primary value stream	Purchasing, manufacturing, marketing, sales
Supporting Process	Supports the core processes	Accounting, recruitment, technical support

Memory Aid

Boss, Builder, Backup. Management = the boss, Operational = the builder of the main value, Supporting = backup.

BPMN

Business Process Model and Notation is a **graphical representation for specifying business processes in a workflow**. The notation is used in a **Business Process Diagram (BPD)**. Goal: support process management for both **technical and business users** with a notation that is intuitive to business people yet can express complex process meaning.

Enterprise Modeling And Data Modeling

Enterprise modeling is the abstract representation, description, and definition of the **structure, processes, information, and resources** of an identifiable business, government body, or other large organization. It means understanding an organization and improving its performance through creating and analyzing enterprise models, including the business processes and the IT used in them.

Data modeling creates a data model using formal data model descriptions and techniques. It is a technique for **defining business requirements for a database**. A data model normally consists of **entity types, attributes, relationships, integrity rules, and definitions**, and is the starting point for interface or database design.

Memory Aid

Data model parts: **E-A-R-I-D** = Entity types, Attributes, Relationships, Integrity rules, Definitions.

Design Diagrams

A design diagram is a **graphic or visual representation of a structure**. It includes **data flow diagrams, structured charts, decision trees**, and other items such as a **cycle of operations (block diagram)**. It shows where data enters a system, where it is processed, the actions taken, and where data leaves as output.

Eight Uses Of A Design Diagram

Use	Meaning
Communications tool	Convenient way to share ideas among engineers, designers, management, and programmers; a concrete visual for complex plans.
Planning tool	Flexible for planning a new system; shows essence and flow lines; analysts see relationships while still planning.
Overview of the system	Shows important elements and relationships without extra details; a bird's-eye view.
Defines roles	Shows the roles of personnel, workstations, and processes; where data, forms, and reports are generated.
Demonstrates relationships	Shows relationships not obvious in words, among data elements and among parts of the system.
Promotes logical procedures	Forces analysts to spell out every path, branch, and flow; logic errors are easier to see visually.
Facilitates troubleshooting	Acts as a blueprint to diagnose breakdowns and communication problems.
Documents the system	Records the key elements clearly and permanently; eases later changes; helps newcomers learn the system.

Memory Aid

One verb each: **Talk, Plan, See, Assign, Connect, Think, Fix, Record.**

Communicate, Plan, Overview, Roles, Relationships, Logic, Troubleshoot, Document.

Data Flow Diagrams (DFDs)

A DFD is a **graphic illustration that shows the flow of data and logic within a system**. The **shape of the symbol** tells the analyst which operation is performed; **arrows** show the **direction** data flows. Labels inside symbols or next to lines describe the flow and transformation. DFD symbols must not be confused with flowchart symbols.

The Four DFD Symbols

Symbol	Shape	What It Is	Notes
Entity	Rectangle	A person or place that provides an input or accepts an output	Source or sink outside the system
Process	Rounded rectangle (sometimes a circle)	An activity or operation; accepts inputs and passes outputs	Numbered sequentially (1.0, 2.0 ...). Shows where calculations are made or information is changed
Data Flow	Arrow	Movement of data between entities, processes, and stores	Each arrow carries a data object (a form, a document, even verbal speech)

Data Store	Open rectangle	Where data is kept permanently or temporarily	Manual store = simple rectangle with an open end. Automated store = open rectangle with an extra vertical line near the closed end. Must be labeled
-------------------	----------------	---	---

Memory Aid

E-P-F-S: Every Person Files Stuff = Entity, Process, Flow, Store.

Shape hooks: Entity = **box** (a thing), Process = **soft box** (it changes things, rounded), Flow = **arrow**, Store = **open shelf** (open side).

Manual vs automated store: **the extra line means automation** (a bar near the closed end).

Watch Out

Functional model says the store is open on the **right**. DFD slide says an **open rectangle**, and a manual store is a rectangle with an **open end**. If asked which store has the extra vertical line: **automated**.

Eight Guidelines For Drawing DFDs

1. **Do not mix levels of detail** on one chart. Draw several DFDs, each at a different level.
2. Pick **one notation**, Gane and Sarson **or** Yourdon and DeMarco, and use it consistently.
3. Use a **template** for uniform symbols in permanent documentation. Freehand is fine for rough or temporary diagrams.
4. **Connect symbols with flow lines**. An arrowhead at one end shows one-way flow; arrowheads at both ends show two-way flow.
5. **Name and label** all symbols and connectors with descriptive names.
6. **Correlate** symbols to other DFDs using reference numbers.
7. **Desk check** every DFD so each symbol is logically connected and describes the flow correctly.
8. **Label the top** of each DFD with the system name, date prepared, preparer's name, and other information.

Memory Aid

Chunk as 3 + 2 + 3:

How to draw (1 to 3): levels, one notation, template.

How to connect and name (4 and 5): flow lines, labels.

How to check and file (6 to 8): correlate, desk check, label the top.

Reading The Slide Examples

The example on the slides is a **production scheduling system**. One version is a single box, **PRODUCTION SCHEDULING SYSTEM**, surrounded by three external entities (**Supplier**,

Customer, Production Department) with flows such as Price Quotation, Specifications, Purchase Order, Price List, Contact, and Report. That is the **big-picture, one-process view**. The other version breaks the system into six numbered processes (1.0 Estimating of Amount, 2.0 Scheduling of Production, 3.0 Preparing of Inventory Report, 4.0 Preparing of Purchase Order, 5.0, 6.0) with data stores such as **Production File** and **Inventory File**. That is the **detailed view**. This is why guideline 1 says not to mix levels.

Chapter 2: System Requirements And Requirements Analysis

System Requirements

System requirements are the specifications a device must have to use certain hardware or software. Example: a computer may need a specific I/O port for a peripheral, or a smartphone may need a specific operating system to run an app. Most software defines **two sets: minimum and recommended**. Requirements tend to **increase over time** as software needs more power.

Example: a video game may run on the minimum CPU and GPU but performs better (better graphics and faster frame rates, **FPS**) on the recommended hardware.

Typical Software Checklist (6)	Typical Hardware Checklist (4)
Operating System (OS)	Operating System (OS)
Minimum CPU or processor speed	Available ports (USB, Ethernet, etc.)
Minimum GPU or video memory	Wireless connectivity (WiFi)
Minimum system memory (RAM)	Minimum GPU (displays and graphics hardware)
Minimum free storage space	
Audio hardware (sound card, speakers)	

Memory Aid

Software list: **O-C-G-R-S-A** = OS, CPU, GPU, RAM, Storage, Audio.

Hardware list is the short one: **O-P-W-G** = OS, Ports, WiFi, GPU.

Flexible Vs Fixed Requirements

- **Not flexible:** the **operating system** and the **disk space** needed to install the software.
- **Can vary a lot** between minimum and recommended: **CPU, GPU, and RAM**.

Memory Aid

OS and Disk are fixed; CPU, GPU, RAM flex. Fixed = "it either fits or it does not".

Requirements Analysis

Requirements analysis covers all tasks in **investigating, scoping, and defining** a new or altered system. The **first activity is the preliminary investigation**, where **data collection** is very important, using **fact-finding techniques**. It is done by **requirements engineers and business analysts**, together with **systems engineers or software developers**, to identify the needs of a client.

Fact-Finding Techniques

Technique	What It Is	Key Points To Remember
Interviews	Analysts collect information about the	Finds misunderstandings, unrealistic expectations, problems, and resistance to

	current system from potential users	the new system. Time consuming.
Record Inspections (Reviews)	Study existing records	Reports, bills, policy manuals, regulations, standard operating procedures
Questionnaires	Collect data from large groups	Two kinds: open-ended and closed-ended (see below)
Observation	Watch the work being done	A skill the analyst must develop: pick the right person, right place, right information; understand how departments and workflows connect
Research	An advanced fact-finding technique	Get information from published materials or documents to check and verify facts

Memory Aid

I Read Questions, Observe, Research = Interview, Records, Questionnaire, Observation, Research.

Interview = slow but rich. Questionnaire = many people. Record review = paper trail.

Observation = watch. Research = check facts in published sources.

Open-Ended Vs Closed-Ended Questionnaires

	Open-Ended	Closed-Ended
Answers	In the person's own words	Choose from a set of prescribed answers
Used to learn	Feelings, opinions, general experiences, process detail or problems	Specific responses
Type of data	Qualitative	Quantitative
Cost note		Costly because the questions must be printed

Memory Aid

Open = Opinions = Qualitative. Closed = Counts = Quantitative.

One-Page Cram Sheet

Topic	Remember
Four modeling kinds	Functional, System Architecture, Business Process, Enterprise (FABE)
Functional model elements	Process, Store, Flow, External Entity (PSFE)
FFBD	Multi-tier, time-sequenced, step-by-step
Architecture	Conceptual model of structure, behavior, views
Business process types	Management, Operational (core), Supporting
BPMN	Notation for workflows, drawn as a Business Process Diagram
Data model parts	Entity types, attributes, relationships, integrity rules, definitions
Design diagrams include	DFDs, structured charts, decision trees, block diagrams
DFD symbols	Entity (rectangle), Process (rounded, numbered), Flow (arrow), Store (open rectangle)
Automated data store	Open rectangle with an extra vertical line near the closed end
DFD notations	Gane and Sarson, or Yourdon and DeMarco: pick one
Requirements sets	Minimum and recommended
Fixed requirements	OS and disk space
Fact-finding	Interviews, records, questionnaires, observation, research